

左偏红黑树

左偏红黑树是红黑树的一种变体，它的对红边（点）的位置做了一定限制，使得其插入与删除操作可以与 2-3 树构成一一对应。

我们假设读者已经至少掌握了一种基于旋转的平衡树，因此本文不会对旋转操作进行讲解。

红黑树

性质

一棵红黑树满足如下性质：

- 1. 节点是红色或黑色；
- 2. NIL 节点（空叶子节点）为黑色；
- 3. 红色的节点的所有儿子的颜色必须是黑色，即从每个叶子到根的所有路径上不能有两个连续红色节点；
- 4. 从任一节点到其子树中的每个叶子的所有简单路径上都包含相同数目的黑色节点。（黑高平衡）

这保证了从根节点到任意叶子的最长路径（红黑交替）不会超过最短路径（全黑）的二倍。从而保证了树的平衡性。

维护这些性质是比较复杂的，如果我们要插入一个节点，首先，它一定会被染色成红色，否则会破坏性质 4。即使这样，我们还是有可能会破坏性质 3。因此需要进行调整。而删除节点就更加麻烦，与插入类似，我们不能删除黑色节点，否则会破坏黑高的平衡。如何方便地解决这些问题呢？

左偏红黑树（Left Leaning Red Black Tree）

解释

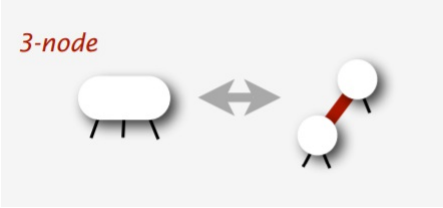
左偏红黑树是一种容易实现的红黑树变体。

在以下左偏红黑树示意图中，是边具有颜色而不是节点具有颜色。我们习惯用一个节点的颜色代指它的父亲边的颜色。

左偏红黑树对红黑树进行了进一步限制，一个黑色节点的左右儿子：

- 要么全是黑色；
- 要么左儿子是红色，右儿子是黑色。

符合条件的情况：



不符合条件的情况：

Disallowed

- right-leaning 3-node representation

standard red-black trees allow this one

- two reds in a row

original version of left-leaning trees used this 4-node representation

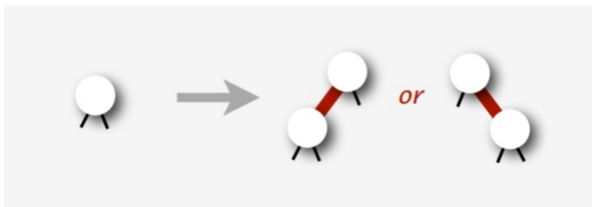
single-rotation trees allow all of these

这是左偏树的「左偏」性质：红色边只能是左偏的。

过程

插入

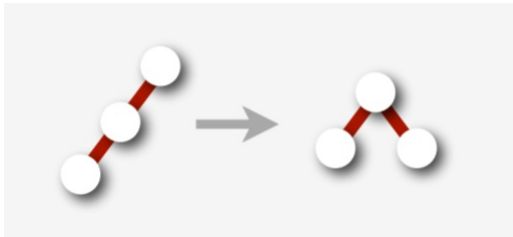
我们首先使用普通的 BST 插入方法，在树的底部插入一个红色的叶子节点，然后通过从下向上的调整，使得插入后的树仍然符合左偏红黑树的性质。下面描述调整的过程：



插入后，可能会产生一条右偏的红色边，因此需要对红边右偏的情况进行一次左旋：



考虑左旋后会产生两条连续的左偏红色边：



因此需要把它进行一次右旋。而对于右旋后的情况，我们应该对它进行 `color_flip`：即翻转该节点和它的两个儿子的颜色



从而消灭右偏的红边。

► 参考代码（部分）

删除

删除操作基于这样的思想：我们不能删除黑色的节点，因为这样会破坏黑高。所以我们需要保证我们最后删除的节点是红色的。

删除最小值节点

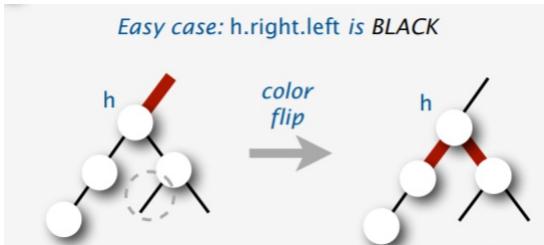
首先来试一下删除整棵树里的最小值。

怎样才能保证最后删除的节点是红色的呢？我们需要在向下递归的过程中保证一个性质：如果当前节点是 h ，那么需要保证 h 是红色，或者 $h \rightarrow lc$ 是红色。

考虑这样做的正确性，如果我们能够通过各种旋转和反转颜色操作成功维护这个性质，那么当我们到达最小的节点 h_{min} 的时候，有 h_{min} 是红色，或者 h_{min} 的左子树——但是 h_{min} 根本没有左子树！所以这就保证了最小值节点一定是红的，既然它是红色的，我们就可以大胆的删除它，然后用与插入操作相同的调整思路对树进行调整。

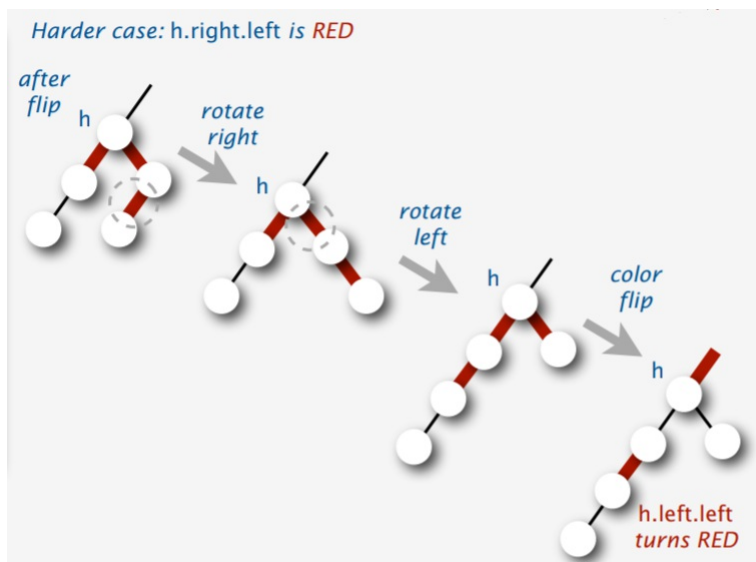
下面我们来考虑怎么满足这个性质，注意，我们会在向下递归的时候 临时地 破坏左偏红黑树的若干性质，但是当我们从递归中返回时还会将其恢复。

如下图所描述的，是一种较为简单的情况，此时 $h \rightarrow rc \rightarrow lc$ 为黑色，我们只需要一次翻转颜色即可：



并且，在如上所示的翻转之后，不会使 $h \rightarrow rc$ 与 $h \rightarrow rc \rightarrow lc$ 形成连续的红边；

但如果 $h \rightarrow rc \rightarrow lc$ 是红色，情况会比较复杂：



如果只进行翻转颜色，会产生连续的红边，而考虑我们递归返回的时候，是无法修复这样的情况，因此需要进行处理。

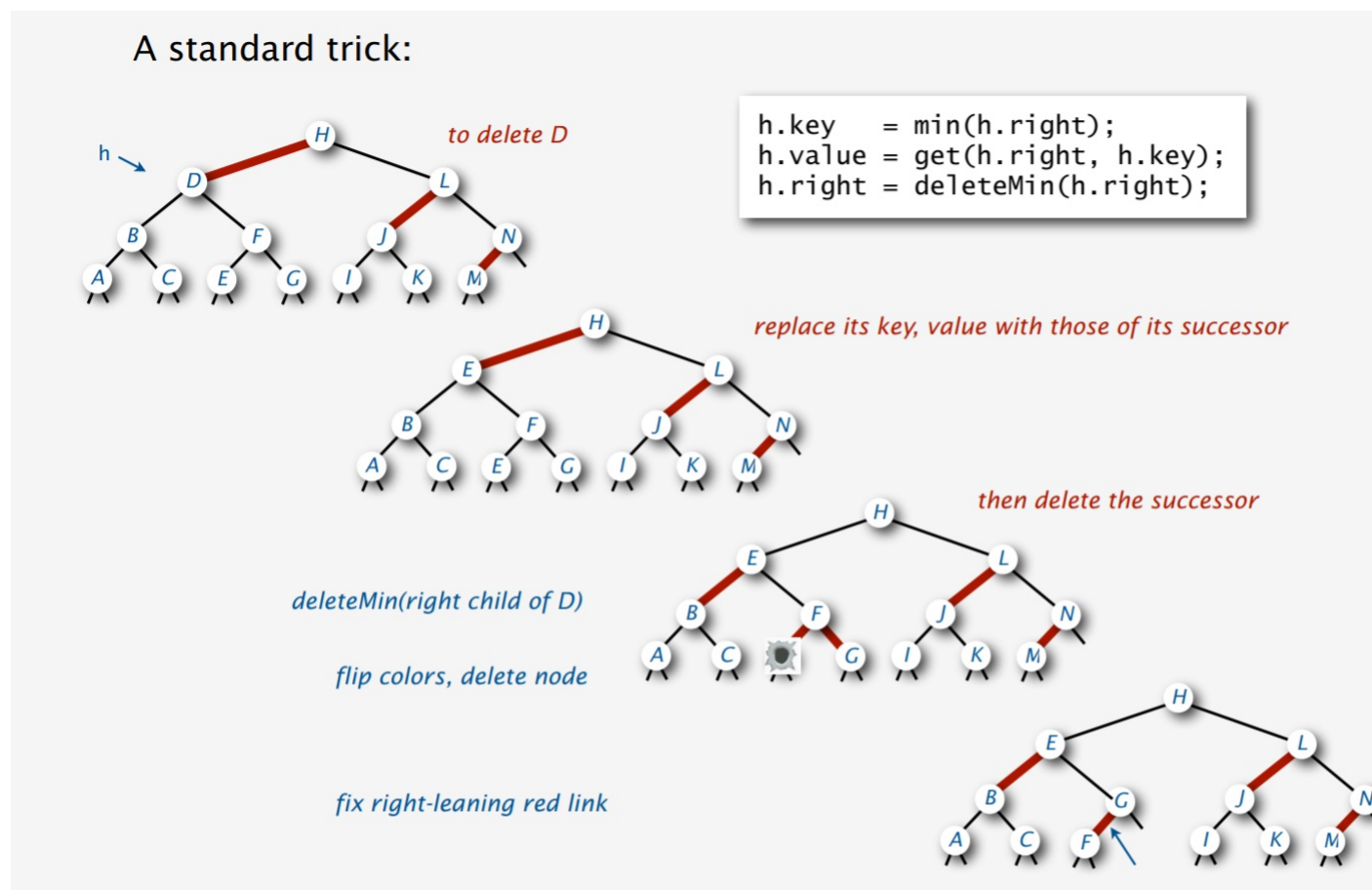
然后就可以进行删除了：

► 参考代码（部分）

删除任意节点

我们首先考虑删除叶子：与删最小值类似，我们在删除任意值的过程中也要维护一个性质，不过这次比较特殊，因为我们不是只向左边走，而是可以向左右两个方向走，因此在删除过程中维护的性质是这样的：如果往左走，当前节点是 h ，那么需要保证 h 是红色，或者 $h \rightarrow lc$ 是红色；如果往右走，当前节点是 h ，那么需要保证 h 是红色，或者 $h \rightarrow rc$ 是红色。这样可以保证我们最后总会删掉一个红色节点。

下面考虑删除非叶子节点，我们只需要找到其右子树（如果有）里的最小节点，然后用右子树的最小节点的值代替该节点的值，最后删除右子树里的最小节点。



那如果没有右子树怎么办？我们需要把左子树旋转过来，这样就不会出现这个问题了。

► 参考代码（部分）

实现

下面的代码是用左偏红黑树实现的 `Set`，即有序不可重集合：

► 参考代码

参考资料与拓展阅读

- [Left-Leaning Red-Black Trees](#)- Robert Sedgewick Princeton University
 - [Balanced Search Trees](#)-_Algorithms_Robert Sedgewick | Kevin Wayne
-

本页面最近更新: 2024/10/9 22:38:42, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [Enter-tainer](#), [giiiiithub](#), [hly1204](#), [iamtwz](#), [lr1d](#), [kigawas](#), [ksyx](#), [luxuryspark567](#), [mgl](#), [orzAtalod](#), [sandyzikun](#), [SunsetGlow95](#), [Tiphereth-A](#)

本页面的全部内容[在 CC BY-SA 4.0 和 SATA 协议之条款下](#)提供, 附加条款亦可能应用